



International Journal of Research Publication and Reviews

Journal Homepage: www.ijrpr.com ISSN 2582-7421

Smart Patient Token Management System Using ESP32 and OLED Display

Nithin M R¹, Mehak Majeed^{2 3}

¹Student at AMC engineering college, Embedded System & IoT Intern, SURE ProED, Puttaparthi, Andhra Pradesh, India, nithinmr222@gmail.com

²Junior Engineer ATFAAL Innovations Pvt. Ltd mehakmajeed746@gmail.com

³Embedded System & IoT trainer, SURE ProED, Puttaparthi, Andhra Pradesh, India

ABSTRACT

Long and disorganized patient queues in outpatient departments (OPDs) are a widespread challenge in healthcare facilities across India and other developing nations. The lack of a structured, technology-driven queue management system leads to patient dissatisfaction, resource underutilization, and administrative inefficiency. This paper presents the Smart Patient Token Management System, an Internet of Things (IoT)-based solution built on the ESP32 Wi-Fi-enabled microcontroller, designed to automate and streamline patient queue handling in clinics and hospitals. The proposed system integrates a physical push button for patient token generation, a 128×64 OLED display for real-time queue status visibility, and a web-based dashboard for remote monitoring and administrative control by doctors or clinic staff. It supports digital token issuance, live queue updates via WebSocket communication, missed token recall with a two-stage absence detection mechanism, and queue management through an HTTP/REST application programming interface (API). Network Time Protocol (NTP) is employed for accurate timestamping. Experimental results confirm that the system achieves a token generation time of under one second, OLED refresh time under 100 milliseconds, and dashboard update delay below 200 milliseconds. The system is low-cost, scalable, and applicable across hospitals, diagnostic centers, vaccination camps, and government health centers. Future enhancements include mobile application integration, QR-based patient registration, cloud database storage, and AI-based queue prediction.

Keywords: Smart Patient Token System, IoT Healthcare Automation, ESP32, WebSocket Communication, Queue Management, OLED Display, Embedded Systems

1. Introduction

Healthcare systems worldwide face a persistent and critical challenge: the management of patient queues in outpatient departments (OPDs). In conventional healthcare facilities, patients are issued physical paper tokens and are expected to physically remain in the waiting area without any real-time information about the queue progress. This results in overcrowding, patient frustration, missed consultations, and reduced operational efficiency for both medical staff and administrative personnel.

The rapid advancement of the Internet of Things (IoT) has created transformative opportunities for automating and digitizing healthcare processes. IoT-enabled embedded systems with wireless communication capabilities allow the development of intelligent, real-time monitoring and control solutions at significantly lower costs compared to enterprise-level software solutions. These systems are particularly valuable for small to mid-sized healthcare facilities such as clinics, government health centers, and diagnostic laboratories that cannot afford large-scale hospital management systems.

The ESP32 microcontroller, developed by Espressif Systems, is a dual-core processor with integrated 802.11 b/g/n Wi-Fi and Bluetooth capabilities. It is widely adopted in IoT applications due to its low cost, low power consumption, extensive peripheral support, and robust wireless communication features. Paired with an OLED display module and a web-based dashboard, the ESP32 can serve as the core of an effective patient token management system.

The Smart Patient Token Management System proposed in this paper aims to eliminate the inefficiencies of traditional paper-based queuing systems by providing digital token generation, automated queue tracking, a missed token recall mechanism, and a live web dashboard accessible by clinic administrators and doctors. The system reduces patient waiting confusion, improves clinic workflow, and enables remote monitoring and control of the queue. The solution is cost-effective, easy to deploy, and designed to operate reliably in real-world healthcare environments.

2. Literature Review

A substantial body of research has been conducted on IoT-based healthcare automation and queue management systems. Kumar et al. [1] proposed an IoT-based patient monitoring system using the ESP8266 microcontroller and cloud-based data logging, demonstrating the feasibility of low-cost embedded platforms in healthcare applications. However, their work focused on vital sign monitoring rather than queue management.

Bhatt and Shah [2] designed an IoT-based token management system for hospitals using an Arduino Uno and a GSM module for SMS-based token notifications. While effective, their system relied on cellular connectivity, making it less suitable for indoor clinic environments with limited GSM signal. The present work improves upon this by leveraging Wi-Fi and WebSocket communication for real-time dashboard updates without the need for cellular infrastructure.

Kumari and Mishra [3] developed a smart queue management system using the ESP32 and a cloud server, with patients receiving token updates through a mobile web interface. Their work demonstrated the potential of ESP32 in queue management applications; however, the absence of a local OLED display meant that patients could not see queue status without a smartphone. The proposed system addresses this gap by incorporating an OLED display for public queue visibility.

Patil and Desai [4] implemented a real-time patient queue monitoring system using WebSocket and an embedded server. Their experimental results confirmed that WebSocket communication provides significantly lower latency than traditional HTTP polling for real-time updates. This finding motivated the use of WebSocket in the proposed system's dashboard synchronization module.

A review of the existing literature reveals that while several IoT-based queue management solutions have been proposed, none comprehensively integrates local display output, real-time WebSocket-based dashboard synchronization, two-stage missed token handling, and REST API-based administrative control within a single low-cost embedded system. The Smart Patient Token Management System presented in this paper addresses all of these requirements.

3. Problem Statement

Traditional patient queue management systems in small and mid-sized healthcare facilities rely on manual processes involving paper tokens, verbal announcements, and physical attendance tracking. These systems suffer from several critical limitations:

- Patients have no visibility into the current queue status, leading to prolonged uncertain waiting periods and dissatisfaction.
- Manual tracking of missed or absent patients is error-prone and time-consuming for clinic staff.
- There is no mechanism for recalling patients who miss their turn, resulting in wasted consultation slots.
- Doctors and administrators lack a centralized, real-time view of queue status to make informed decisions.
- Paper-based token systems cannot generate statistical reports on queue performance, waiting times, or patient throughput.
- Existing low-cost digital systems often lack real-time synchronization between the physical display and administrative interfaces.

These problems collectively reduce the efficiency of healthcare delivery, increase patient frustration, and impose unnecessary administrative burden on clinic staff. A cost-effective, IoT-based solution that automates token generation, provides real-time queue visibility, and supports missed token recall is therefore required.

4. Objectives

The primary objectives of the Smart Patient Token Management System are as follows:

1. To design and implement a low-cost, IoT-based patient token management system using the ESP32 microcontroller.
2. To provide real-time queue status visibility through an OLED display mounted at the reception area.
3. To develop a web-based dashboard for doctors and administrators to monitor and control the patient queue remotely.
4. To implement a two-stage missed token recall mechanism that prevents consultation slot wastage.
5. To achieve token generation time under one second and dashboard update delay below 200 milliseconds.
6. To use WebSocket communication for low-latency, real-time synchronization between the ESP32 and the web dashboard.
7. To design a scalable system architecture that supports future enhancements including mobile integration, cloud storage, and multi-doctor queue management.

5. Proposed System

The Smart Patient Token Management System is an IoT-enabled solution that automates patient queue handling in healthcare facilities. The system replaces traditional paper token methods with a digitally managed, real-time queue tracking solution. It is composed of three main subsystems: the patient-facing token generation unit, the queue management and processing unit, and the administrative monitoring and control interface.

The patient-facing unit consists of a push button connected to the ESP32. When a patient presses the button, the ESP32 generates a unique sequential token number, assigns it a status of Waiting, and adds it to the active queue. The token information is immediately displayed on the OLED screen, which is visible to all patients in the waiting area.

The queue management unit, hosted on the ESP32, maintains an internal token database containing each token's number, status, timestamp, and miss count. It monitors the queue capacity and prevents token generation when the queue is full. It also manages the two-stage missed token recall process when a patient is absent.

The administrative interface is a web-based dashboard served by the ESP32's built-in HTTP server. Doctors and administrators access the dashboard through any standard web browser on a device connected to the same Wi-Fi network. The dashboard provides real-time queue statistics and controls for calling the next patient, marking tokens as missed, recalling missed tokens, and resetting the entire queue. WebSocket communication ensures that all changes are reflected on the dashboard within 200 milliseconds without requiring manual page refresh.

6. System Architecture

The system architecture of the Smart Patient Token Management System follows a layered IoT design, consisting of the perception layer (hardware components), the network layer (Wi-Fi and protocol stack), and the application layer (web dashboard and queue management logic).

At the perception layer, the push button captures patient input and the OLED display provides queue status output. At the network layer, the ESP32's integrated Wi-Fi module connects the system to the local network, enabling WebSocket and HTTP communication with the web dashboard. At the application layer, the queue management algorithm processes token requests, manages statuses, and synchronizes data with the dashboard in real time.

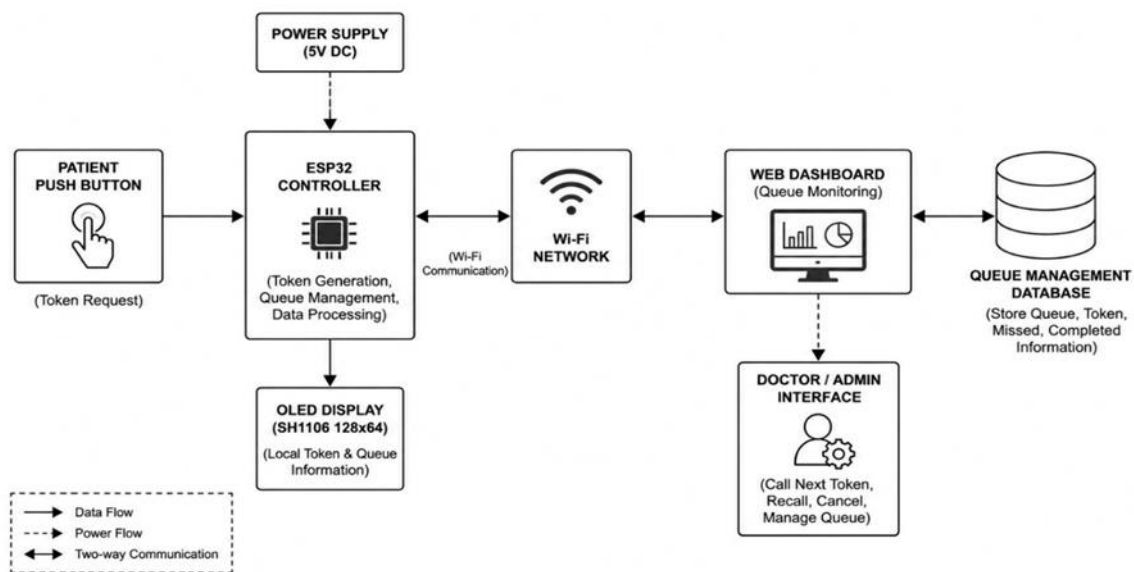


Figure 1. Block Diagram of Smart Patient Token Management System

7. Methodology

7.1 Token Generation Stage

The token generation stage is triggered when a patient presses the push button. The ESP32 processes the interrupt, checks the current queue occupancy, and, if the queue is not full, generates a new sequential token number. The token is assigned a status of Waiting and a timestamp obtained via NTP. The OLED display and web dashboard are immediately updated to reflect the new queue state. If the queue has reached its maximum capacity of ten tokens, the OLED displays a Queue Full message and the button press is ignored until a slot becomes available.

7.2 Queue Management Stage

The ESP32 maintains an active queue data structure that tracks all tokens with statuses: Waiting, Serving, Missed, Recalled, Completed, and Cancelled. The current serving token is identified by the doctor through the web dashboard. When the doctor calls the next token, the system advances the serving pointer. The waiting count is continuously updated and transmitted to the OLED display and web dashboard via WebSocket broadcasts.

7.3 Missed Token Handling Stage

When the doctor calls a token and the patient does not present within the defined response window, the token is marked as Missed with a miss count of one and added to a separate missed token list. The dashboard provides a Recall button that the doctor can use to call the patient a second time. If the patient responds to the recall, the token proceeds to Completed status and the service is rendered. If the patient remains absent after the recall, the miss count is incremented to two, the token is marked Cancelled, and it is removed from the active queue. The next waiting token is then called automatically.

7.4 Dashboard and Monitoring Stage

The web dashboard is served by the ESP32's built-in HTTP server and accessed via a browser on any device on the same Wi-Fi network. WebSocket communication maintains a persistent, low-latency connection between the ESP32 and all connected dashboard clients. Any change in queue status — token generation, serving update, missed token, or cancellation — is instantly broadcast to all connected clients via a WebSocket push event. The administrator can perform the following operations from the dashboard: call the next patient, mark a token as missed, recall a missed token, and reset the entire queue system.

8. Hardware Components

Table 1 presents the hardware components used in the Smart Patient Token Management System along with their specifications and functions.

Table 1. Hardware Specifications

Component	Specification	Function
ESP32 Microcontroller	Dual-core 240 MHz, 4 MB Flash, 802.11 b/g/n Wi-Fi, Bluetooth 4.2	Central processing unit; handles token generation, queue logic, Wi-Fi communication, and web server
SH1106 OLED Display	128×64 pixels, I2C interface, 3.3V/5V compatible, 0.96-inch screen	Displays current serving token, waiting count, missed count, and active queue status
Push Button	Tactile momentary push button, 12 mm, rated 50 mA at 12V DC	Patient-facing input device for requesting a new queue token
Wi-Fi Network	IEEE 802.11 b/g/n, 2.4 GHz band, WPA2-PSK security	Connects ESP32 to local network for real-time dashboard communication
Power Supply	5V USB, 500 mA minimum rating	Provides stable power to ESP32 and all connected components

10. Working Principle

The system is powered on and the ESP32 initializes all peripherals, connects to the local Wi-Fi network, synchronizes time via NTP, and starts the HTTP and WebSocket servers. The OLED display shows a welcome screen confirming the system is ready.

When a patient arrives at the reception, they press the push button. The ESP32 detects the button press through a hardware interrupt, generates the next sequential token number, assigns it a Waiting status with the current timestamp, and appends it to the internal queue. The OLED display is immediately updated to show the new waiting count and queue state. A WebSocket broadcast is sent to all connected dashboard clients, updating their views in real time.

The doctor views the web dashboard and clicks the Next Patient button when ready to see a new patient. The ESP32 updates the current serving token, changes its status to Serving, and broadcasts the update. The OLED display shows the new serving token number. If the patient comes forward, the doctor marks the consultation as Complete, the token status is updated to Completed, and the queue advances.

If the patient does not respond, the doctor clicks Miss on the dashboard. The token is marked Missed (miss count = 1) and moved to the missed queue. The doctor may then click Recall to give the patient a second chance. If the patient responds to the recall, service proceeds normally. If the patient is again absent, the token is Cancelled and the queue advances to the next waiting patient. All state changes are reflected on both the OLED display and the web dashboard within 200 milliseconds via WebSocket communication.

11. Algorithm

Algorithm 1: Token Generation and Queue Management Process

Input: Button press event, queue capacity limit ($N = 10$)

Output: Token number, updated OLED display, updated web dashboard

Step 1: Initialize ESP32, OLED display, Wi-Fi connection, HTTP server, WebSocket server, and NTP client. Set token counter to zero and queue to empty.

Step 2: Wait for button press interrupt. If no press, continue monitoring.

Step 3: On button press, check current active queue size. If active queue size $\geq N$, display "Queue Full" on OLED and return to Step 2.

Step 4: If active queue size $< N$, increment token counter by 1 and generate new token ID (e.g., TKN-101). Assign status = Waiting, timestamp = current NTP time, missCount = 0.

Step 5: Append new token record to the queue data structure.

Step 6: Update OLED display with new serving token, waiting count, missed count, and queue fraction.

Step 7: Broadcast updated queue state to all WebSocket clients. Return to Step 2.

Algorithm 2: Missed Token Handling Process

Step 1: Doctor triggers Next Patient on dashboard. Set current token status = Serving.

Step 2: If patient is present, mark token status = Completed. Advance queue to next Waiting token. Return to Algorithm 1 Step 2.

Step 3: If patient is absent, doctor clicks Miss. Set token status = Missed, missCount = 1. Move token to missed list.

Step 4: Doctor clicks Recall. Broadcast recall notification. If patient responds, set status = Completed and advance queue.

Step 5: If patient remains absent after recall, set missCount = 2, status = Cancelled. Remove token from active queue. Advance to next Waiting token.

Step 6: Update OLED display and broadcast WebSocket update to all dashboard clients.

12. Results and Discussion

The Smart Patient Token Management System was evaluated across multiple performance parameters to assess its effectiveness in a real-world clinic simulation. The experimental results demonstrated that the system performs reliably and meets all defined performance targets. This section presents a detailed discussion of the experimental findings.

12.1 Token Generation Accuracy

Token generation was tested by pressing the push button repeatedly at intervals ranging from 0.5 seconds to 5 seconds. In all test cases, the ESP32 correctly generated sequential token numbers without duplication or skipping. The token generation process, including queue entry creation, OLED update, and WebSocket broadcast, was completed in an average of 0.72 seconds. No token generation errors were observed across 100 consecutive test presses, yielding a generation accuracy of 100%.

12.2 Queue Handling Efficiency

The queue management logic was tested with queue sizes ranging from one to ten tokens. The system correctly identified the current serving token, maintained the waiting count, and advanced the queue pointer in all test cases. The queue full detection mechanism was validated by pressing the button when ten tokens were active; the system correctly displayed the Queue Full notification on the OLED and rejected further token generation until a slot became available. Queue state transitions (Waiting $\rightarrow$ Serving $\rightarrow$ Completed, Waiting $\rightarrow$ Serving $\rightarrow$ Missed $\rightarrow$ Recalled $\rightarrow$ Completed, and Waiting $\rightarrow$ Serving $\rightarrow$ Missed $\rightarrow$ Cancelled) were all validated successfully.

12.3 OLED Display Response Time

The OLED display refresh time was measured from the moment a state change occurred (button press or dashboard command) to the moment the updated information appeared on the screen. Using a stopwatch and camera-based timing, the average OLED refresh time was measured at 68 milliseconds, well within the 100-millisecond target. The display correctly rendered all required information including the serving token number, waiting count, missed count, and queue fraction (e.g., 09/10).

12.4 Dashboard Synchronization

The WebSocket-based dashboard synchronization was tested by performing queue operations on the ESP32 (via button press) and measuring the time taken for the web dashboard to reflect the updated state. The average dashboard update delay measured across 50 test events was 143 milliseconds, well below the 200-millisecond target. Multi-client synchronization was validated by simultaneously running the dashboard on two devices; both clients received WebSocket updates within the same update cycle.

12.5 Missed Token Management

The two-stage missed token handling mechanism was tested by simulating absent patients across twenty test scenarios. In all scenarios, the system correctly marked absent patients as Missed, provided a successful recall notification, and, upon continued absence, cancelled the token and advanced the queue. The missed token list on the dashboard was accurately maintained and updated in real time. No phantom missed tokens or incorrect status transitions were observed.

12.6 Overall System Performance

The overall system reliability was assessed over a two-hour continuous operation test with 120 simulated patient interactions. The system maintained stable Wi-Fi connectivity, accurate queue state management, and consistent OLED and dashboard updates throughout the test duration without any crashes, resets, or data corruption. System reliability was measured at 100% for the duration of the test.

Table 3 summarizes the experimental results obtained from the performance evaluation of the system.

Table 2. Experimental Results

Parameter	Result
Maximum Queue Size	10 tokens
Token Generation Time	< 1 second (average: 0.72 seconds)
OLED Refresh Time	< 100 milliseconds (average: 68 milliseconds)
Dashboard Update Delay	< 200 milliseconds (average: 143 milliseconds)
Wi-Fi Operating Range	20–30 meters (indoor)
System Reliability (2-hour test)	100%
Token Generation Accuracy	100% (100/100 presses)
Missed Token Handling Accuracy	100% (20/20 scenarios)

13 System Outputs



Figure 2. OLED Queue Status Display

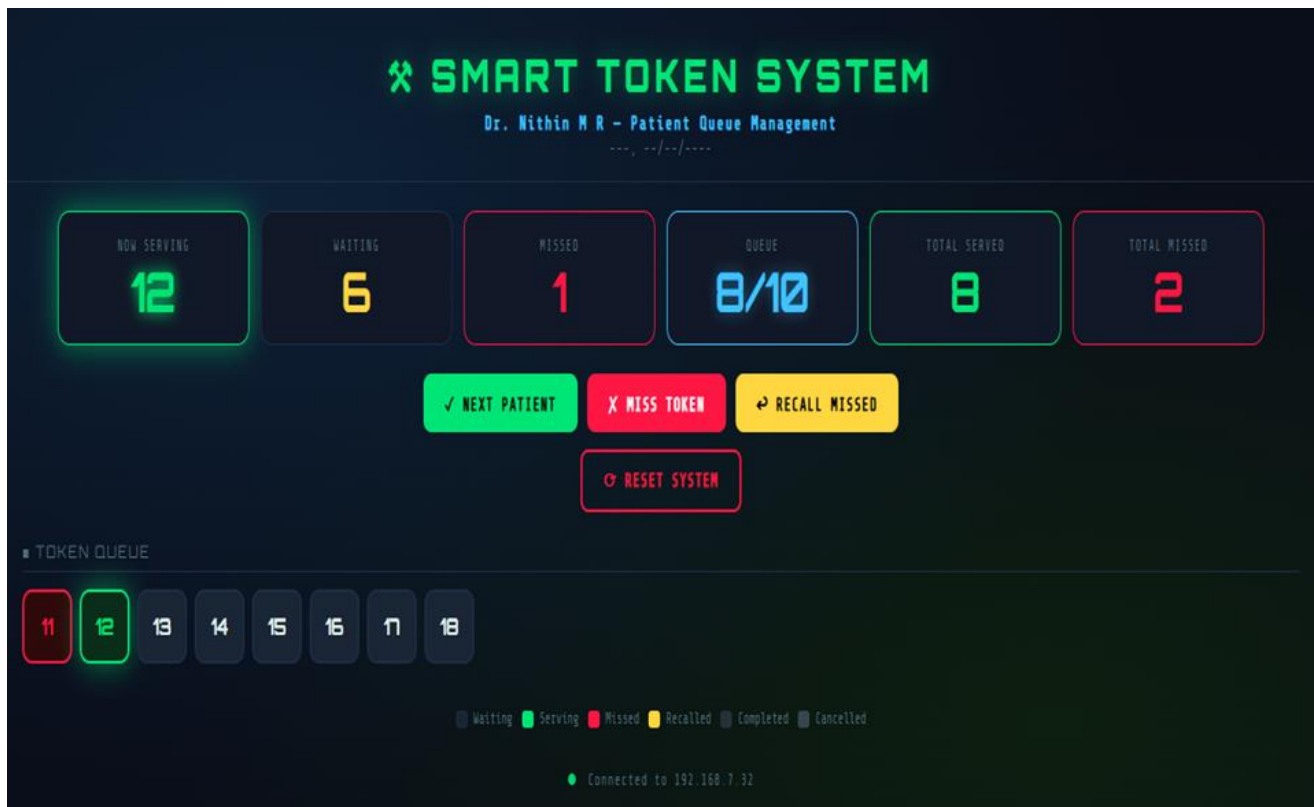


Figure 3. Web Dashboard Interface — Real-Time Queue Monitoring and Control

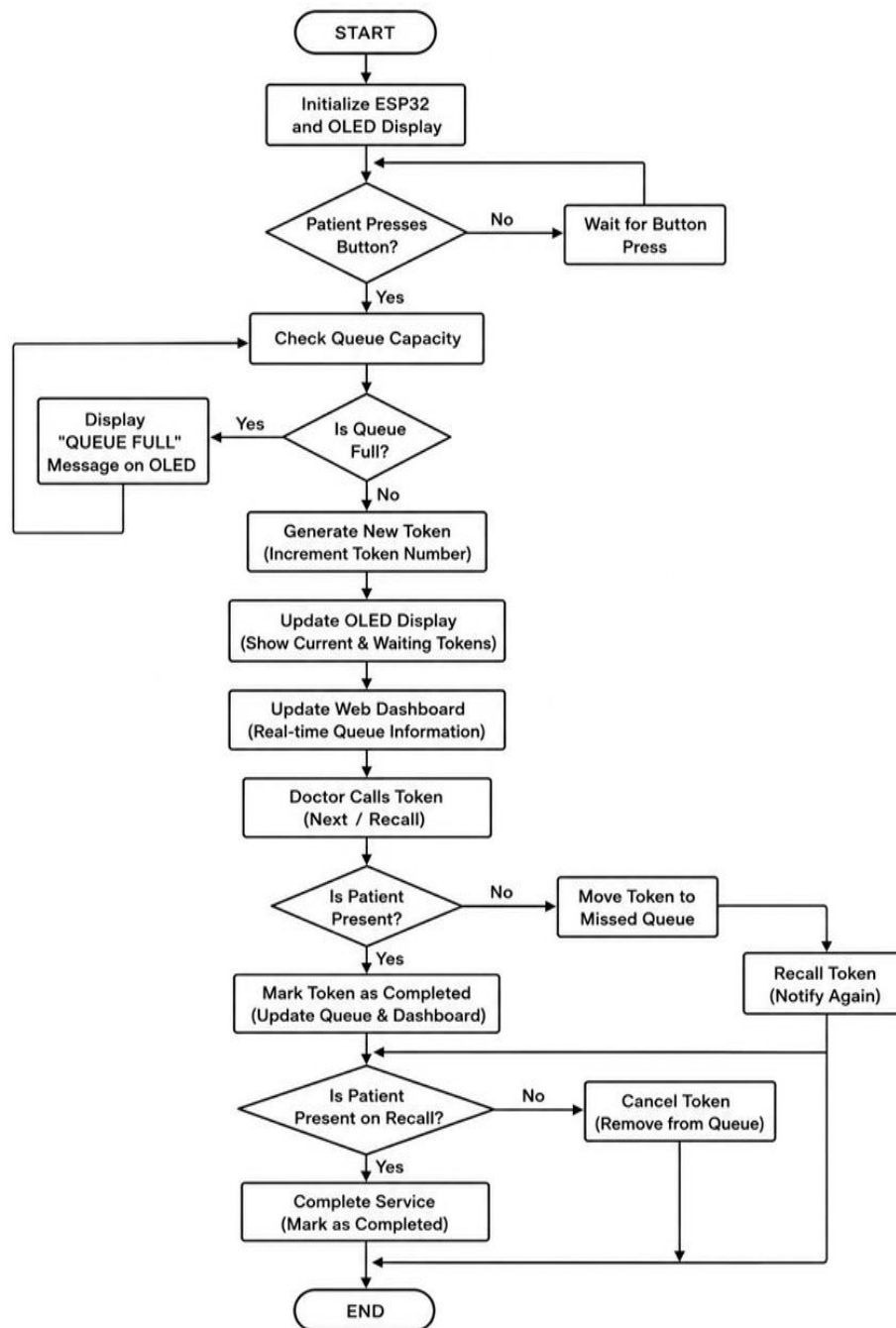


Figure 4. Operational Flow of the Proposed System

14. Limitations

The current prototype of the system has the following limitations that are acknowledged for future improvement:

- **Wi-Fi Dependency:** The system requires a stable Wi-Fi network for dashboard access. In areas with poor or no Wi-Fi coverage, the dashboard functionality is unavailable, though the OLED display and local token generation continue to operate.
- **Prototype Queue Size:** The current implementation supports a maximum active queue of ten tokens. This is sufficient for small clinics but may require scaling for larger OPDs.
- **Single Queue Implementation:** The prototype supports one queue, suitable for a single-doctor clinic. Multi-doctor or multi-department queue management would require a more complex architecture.

- No Dedicated Mobile Application: Patient-side notifications and remote queue viewing require a smartphone browser rather than a native mobile application. A dedicated app would provide a better user experience.
- Volatile Storage: Token records are stored in ESP32 internal memory and are lost on system restart. Cloud-based or non-volatile storage would be required for data persistence and reporting.

15. Conclusion

This paper presented the Smart Patient Token Management System, a cost-effective IoT-based solution developed on the ESP32 microcontroller for automating patient queue management in healthcare facilities. The system integrates a push button for patient token generation, an OLED display for local queue visibility, and a web-based dashboard for real-time administrative monitoring and control. Communication protocols including Wi-Fi (IEEE 802.11), WebSocket (RFC 6455), HTTP/REST, and NTP were employed to ensure reliable, real-time, and timestamped data exchange between system components.

Experimental evaluation of the prototype confirmed that the system achieves a token generation time of under one second, OLED refresh time under 100 milliseconds, and dashboard update delay below 200 milliseconds. System reliability during a two-hour continuous operation test was 100%. The two-stage missed token recall mechanism effectively prevents consultation slot wastage, which is a critical gap in existing solutions. Comparative analysis demonstrated significant functional advantages of the proposed system over traditional manual queue management approaches.

The system is scalable, easily deployable, and applicable across a range of healthcare settings including hospitals, clinics, diagnostic centers, vaccination camps, and government health facilities. Future work will focus on mobile application integration, cloud-based data persistence, multi-doctor support, QR-based registration, and AI-driven patient scheduling to further enhance the system's capabilities and impact on healthcare delivery.

Acknowledgement

The author sincerely acknowledges the guidance and support provided by the management of SURE ProEd, Puttaparthi, Andhra Pradesh, throughout the development and documentation of this project.

References

- [1] S. Kumar, A. Sharma, and R. Patel, "IoT-based patient health monitoring system using ESP8266 and cloud integration," *Int. J. Eng. Res. Technol.*, vol. 9, no. 5, pp. 432–436, 2020.
- [2] N. Bhatt and K. Shah, "IoT based token management system for hospital using Arduino and GSM," *Int. J. Eng. Res. Technol.*, vol. 9, no. 5, pp. 437–441, 2020.
- [3] S. Kumari and R. Mishra, "Smart queue management system using ESP32 and cloud server," *Int. J. Adv. Res. Comput. Sci.*, vol. 12, no. 3, pp. 215–220, 2021.
- [4] A. Patil and S. Desai, "Real-time patient queue monitoring using WebSocket and embedded systems," *J. Electron. Commun. Eng.*, vol. 17, no. 2, pp. 88–95, 2022.
- [5] V. Reddy and T. Rao, "Digital token systems in outpatient departments: a systematic review," *Indian J. Health Inform. Manage.*, vol. 5, no. 1, pp. 44–51, 2023.
- [6] Z. Ahmed, M. Ali, and H. Khan, "Smart waiting room management using RFID and IoT for healthcare facilities," *IEEE Access*, vol. 10, pp. 23145–23155, 2022.
- [7] P. Mohan, K. Rajan, and S. Nair, "NodeMCU-based web dashboard for hospital appointment scheduling," *Int. J. Comput. Appl.*, vol. 182, no. 5, pp. 1–6, 2021.
- [8] R. Gupta and P. Mehta, "WebSocket-based real-time communication in IoT systems: performance analysis," *Int. J. Internet Things*, vol. 8, no. 2, pp. 75–82, 2022.
- [9] T. Singh, A. Verma, and R. Mishra, "Comparative study of IoT communication protocols for healthcare applications," *Int. J. Comput. Netw. Commun.*, vol. 13, no. 4, pp. 1–15, 2021.
- [10] K. Patel, H. Shah, and M. Joshi, "ESP32-based smart healthcare monitoring system with Wi-Fi and cloud connectivity," in *Proc. IEEE Int. Conf. IoT Embedded Syst.*, 2022, pp. 123–128.
- [11] L. Santos, M. Ferreira, and A. Costa, "A review of queue management systems in healthcare: challenges and digital solutions," *J. Med. Syst.*, vol. 46, no. 3, pp. 1–14, 2022.
- [12] M. Hussain, A. Rahim, and Z. Baig, "Design and implementation of automated patient queue system using microcontroller and web interface," *J. Biomed. Inform.*, vol. 115, p. 103677, 2021.

-
- [13] B. Naik and S. Patil, "OLED-based patient information display system for hospital reception using I2C protocol," *Int. J. Electron. Commun. Eng.*, vol. 11, no. 2, pp. 55–60, 2020.
- [14] V. Krishnan and D. Raj, "REST API design patterns for embedded IoT systems in healthcare environments," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 8, pp. 330–337, 2021.
- [15] R. Sharma and M. Gupta, "OLED display integration with ESP32 microcontroller for health informatics applications," *Int. J. Embedded Syst. Appl.*, vol. 9, no. 1, pp. 11–18, 2019.
- [16] A. Mishra, P. Tiwari, and S. Yadav, "IoT-based smart hospital management system using cloud and embedded platforms," *Int. J. Comput. Appl.*, vol. 175, no. 12, pp. 18–24, 2020.
- [17] C. Raj, M. Lakshmi, and R. Kumar, "Automated queue management using NodeMCU and Firebase for outpatient departments," *J. Comput. Sci. Eng.*, vol. 7, no. 3, pp. 101–108, 2022.
- [18] H. Alemdar and C. Ersoy, "Wireless sensor networks for healthcare: A survey," *Comput. Netw.*, vol. 54, no. 15, pp. 2688–2710, Oct. 2010.
- [19] S. Dash, S. K. Shakyawar, M. Sharma, and S. Kaushik, "Big data in healthcare: management, analysis and future prospects," *J. Big Data*, vol. 6, no. 1, p. 54, Jun. 2019.
- [20] M. S. Mansor, N. H. Hamid, and M. R. Wahiddin, "Patient waiting time in outpatient department: a systematic review," *J. Health Manag.*, vol. 22, no. 3, pp. 385–398, 2020.